06_supervised_learning

October 2, 2026

1 06 — Métodos de Machine Learning para Construcción de Portafolio

Proyecto: Optimización de Portafolios con ML — Tecnológico de Monterrey, AD2026
Etapa: 06 / 09 — Supervised Learning para Portafolios
Datos: `data/processed/features_clean.parquet` (universo final procesado)

1.1 Objetivo

Construir portafolios de inversión usando métodos supervisados de ML que **predigan retornos futuros** y **transformen esas predicciones en pesos del portafolio**, superando el baseline Markowitz.

1.2 Pipeline general

```
features_clean.parquet

        1. EDA rápido de features disponibles
        2. Ingeniería de target (retorno forward 21d)
        3. División Walk-Forward (CV temporal)

        MODELOS DE PREDICCIÓN DE RETORNOS:
              4a. Ridge / Lasso Regression
              4b. Random Forest Regressor
              4c. XGBoost Regressor

        5. Señales de ranking → pesos del portafolio (long-only)
        6. Backtesting mensual y métricas OOS
        7. Comparativa vs Markowitz baseline
```

1.3 División temporal

Período	Rango	Uso
Train inicial	2015-01-02 → 2019-12-31	Primera ventana de entrenamiento
Walk-Forward	Ventana expandida, refit mensual	CV fuera de muestra real

Período	Rango	Uso
OOS final	2023-01-01 → 2025-03-13	Evaluación final vs benchmark

---

1.4 0. Configuración

[ ]:
```
import sys, os
PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import matplotlib.dates as mdates
import matplotlib.patches as mpatches
import seaborn as sns
from matplotlib.gridspec import GridSpec

# ML
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import cross_val_score
import xgboost as xgb

# Métricas de portafolio
from src.models.markowitz_benchmark import (
    full_metrics,
    compute_portfolio_returns,
    ALL_TICKERS,
    REPRES,
    SECTOR_LOOKUP,
    BENCHMARK,
    RF_ANNUAL,
    TRADING_DAYS,
    max_drawdown,
    annualized_return,
    annualized_vol,
```

```
    sharpe_ratio,
)

#   Estética
plt.rcParams.update({
    'figure.dpi':        120,
    'axes.spines.top':   False,
    'axes.spines.right': False,
    'font.family':       'sans-serif',
    'axes.titlesize':    13,
    'axes.labelsize':    11,
})

PALETTE = {
    'Information Technology': '#4C9BE8',
    'Consumer Staples':       '#2ECC71',
    'Health Care':            '#E74C3C',
    'Consumer Discretionary': '#F39C12',
    'SPY / Benchmark':        '#95A5A6',
}
MODEL_COLORS = {
    'Ridge':         '#3498DB',
    'Lasso':         '#9B59B6',
    'RandomForest':  '#27AE60',
    'XGBoost':       '#E74C3C',
    'EW (baseline)': '#E67E22',
    'SPY':           '#95A5A6',
}

#   Rutas
FEATURES_PATH = '../data/processed/features_clean.parquet'
PRICES_PATH   = '../data/processed/prices_repr.parquet'
CSV_PATH      = '../data/datos.csv'
OUTPUT_DIR    = '../data/processed/ml_portfolio'
os.makedirs(OUTPUT_DIR, exist_ok=True)

#   Parámetros
FECHA_INICIO  = '2015-01-02'
TRAIN_END_ML  = '2019-12-31'   # primera ventana de entrenamiento
OOS_START     = '2020-01-01'
FINAL_OOS     = '2023-01-01'   # comparativa final
HORIZON       = 21             # días de retorno forward (1 mes)
TOP_K         = 10             # número de activos en portafolio long-only
REBALANCE_FREQ = 'ME'          # rebalanceo mensual

print('Configuracion lista')
print(f'  Features: {FEATURES_PATH}')
```

```
print(f'  Horizonte de prediccion: {HORIZON} dias ({HORIZON//21} mes)')
print(f'  Top-K activos: {TOP_K}')
print(f'  Rebalanceo: mensual')
```

1.5 1. Carga y Exploración de Features

[ ]:
```
#   Carga de features procesadas
features_raw = pd.read_parquet(FEATURES_PATH)
print(f'Features shape: {features_raw.shape}')
print(f'Columnas       : {list(features_raw.columns[:10])} ...')
print(f'Tipos de datos :\n{features_raw.dtypes.value_counts()}')
print(f'\nPrimeras filas:')
features_raw.head(3)
```

[ ]:
```
#   Inspeccionar estructura: ¿long o wide?
print('Tipo de indice:', features_raw.index.dtype)
print('Columnas disponibles:', features_raw.columns.tolist())

# Detectar si hay columna 'ticker' o 'symbol' para estructura long
id_cols = [c for c in features_raw.columns if c.lower() in ['ticker', 'symbol',␣
 ↪'asset', 'stock']]
print('\nPosibles columnas de ticker:', id_cols)

# Detectar si el índice es MultiIndex
print('MultiIndex:', isinstance(features_raw.index, pd.MultiIndex))
```

[ ]:
```
#   Normalizar estructura a formato long: (Date, Ticker) como índice

if isinstance(features_raw.index, pd.MultiIndex):
    # Ya está en formato long con MultiIndex
    features = features_raw.copy()
    features.index.names = ['Date', 'Ticker']
    print('Estructura: MultiIndex (Date, Ticker)')

elif len(id_cols) > 0:
    # Columna de ticker identificada
    ticker_col = id_cols[0]
    date_col   = [c for c in features_raw.columns if c.lower() in ['date',␣
 ↪'fecha', 'time']]
    if date_col:
        features = features_raw.set_index([date_col[0], ticker_col])
    else:
        features = features_raw.set_index([features_raw.index, ticker_col])
    features.index.names = ['Date', 'Ticker']
    print(f'Estructura: columnas {id_cols} → MultiIndex (Date, Ticker)')
```

```
elif all('__' in c for c in features_raw.columns):
    # Formato wide con convención feature__ticker → transformar a MultiIndex␣
 ↪(Date, Ticker)
    tuples = [tuple(c.split('__', 1)) for c in features_raw.columns]
    features_copy = features_raw.copy()
    features_copy.columns = pd.MultiIndex.from_tuples(tuples, names=['feature',␣
 ↪'Ticker'])
    features = features_copy.stack(level='Ticker')
    features.index.names = ['Date', 'Ticker']
    print(f'Estructura transformada: MultiIndex (Date, Ticker) con shape␣
 ↪{features.shape}')

else:
    # Intentar inferir: si las columnas son tickers (wide format)
    print('Estructura inferida: wide (fechas x features*tickers o fechas x␣
 ↪tickers)')
    features = features_raw.copy()
    print('NOTA: Se intentará detectar el formato en celdas siguientes')

print(f'\nShape final: {features.shape}')
features.head(3)
```

[ ]:
```
#   Estadísticas descriptivas de features
desc = features.describe().T
print('Estadisticas descriptivas de features:')
print(desc[['count','mean','std','min','25%','50%','75%','max']].round(4).
 ↪to_string())
```

[ ]:
```
#   Missing values
missing = features.isna().mean().sort_values(ascending=False)
missing_nonzero = missing[missing > 0]

print(f'Total features evaluadas : {len(missing)}')
print(f'Features con missing > 0 : {len(missing_nonzero)}')

if len(missing_nonzero) > 0:
    print('\nMissing values por feature (top 20):')
    print(missing_nonzero.head(20).to_string())

    fig, ax = plt.subplots(figsize=(12, 4))
    missing_nonzero.plot(kind='bar', ax=ax, color='#E74C3C', alpha=0.8)
    ax.set_title('Proporcion de Missing Values por Feature', fontweight='bold')
    ax.set_ylabel('Fraccion missing')
    ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
    ax.grid(axis='y', alpha=0.3)
    plt.xticks(rotation=45, ha='right')
```

```
    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/missing_values.png', dpi=120,␣
 ↪bbox_inches='tight')
    plt.show()
else:
    print('\nExcelente: No se encontraron valores faltantes en el dataset de␣
 ↪features (0% missing).')
    fig, ax = plt.subplots(figsize=(8, 2.5))
    ax.text(0.5, 0.5, '0% Missing Values\nDataset completamente limpio (100%␣
 ↪completo)',
            ha='center', va='center', fontsize=12, fontweight='bold',␣
 ↪color='#27AE60')
    ax.axis('off')
    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/missing_values.png', dpi=120,␣
 ↪bbox_inches='tight')
    plt.show()
```

1.6 2. Ingeniería del Target y Selección de Features

[ ]:
```
#   Cargar precios para construir retornos forward
df_raw = pd.read_csv(CSV_PATH)
business_days = pd.bdate_range(start=pd.Timestamp(FECHA_INICIO),␣
 ↪periods=len(df_raw))
df_raw.index = business_days
df_raw.index.name = 'Date'
df_raw = df_raw.drop(columns=['promedio'], errors='ignore')

use_tickers = ALL_TICKERS + [BENCHMARK]
available   = [t for t in use_tickers if t in df_raw.columns]
prices      = df_raw[available].copy()

log_ret     = np.log(prices / prices.shift(1)).dropna(how='all')

# Retorno forward de HORIZON días
fwd_ret = log_ret[ALL_TICKERS].rolling(HORIZON).sum().shift(-HORIZON)

print(f'Retornos forward {HORIZON}d:')
print(f'  Shape: {fwd_ret.shape}')
print(f'  Missing (por el shift): {fwd_ret.isna().sum().mean():.0f} filas/
 ↪activo')
```

[ ]:
```
#   Definir features disponibles para el modelo
# Intentamos detectar columnas numéricas relevantes del parquet
numeric_cols = features.select_dtypes(include=[np.number]).columns.tolist()
```

```
# Excluir columnas que son target o que generan data leakage
EXCLUDE_PATTERNS = ['fwd', 'forward', 'target', 'future', 'next']
FEATURE_COLS = [c for c in numeric_cols
                if not any(p in c.lower() for p in EXCLUDE_PATTERNS)]

print(f'Features candidatas: {len(FEATURE_COLS)}')
print(f'Ejemplos: {FEATURE_COLS[:15]}')
```

[ ]:
```
#   Construir dataset de ML en formato (Date, Ticker, features..., target)
# Necesitamos unir features con target forward

if isinstance(features.index, pd.MultiIndex):
    # Formato long: unir directamente
    fwd_long = fwd_ret.stack().rename('target_fwd').reset_index()
    fwd_long.columns = ['Date', 'Ticker', 'target_fwd']
    fwd_long = fwd_long.set_index(['Date', 'Ticker'])

    ml_df = features[FEATURE_COLS].join(fwd_long, how='inner')
    ml_df = ml_df.dropna(subset=['target_fwd'])

    print(f'Dataset ML (long): {ml_df.shape}')
    print(f'Fechas unicas: {ml_df.index.get_level_values("Date").nunique()}')
    print(f'Tickers unicos: {ml_df.index.get_level_values("Ticker").nunique()}')

else:
    # Formato wide: hacer stack de features y unir
    print('Formato wide detectado - construyendo ML dataset desde precios...')

    # Construir features básicas desde retornos directamente
    activos = [t for t in ALL_TICKERS if t in log_ret.columns]
    spy_ret = log_ret[BENCHMARK] if BENCHMARK in log_ret.columns else None

    frames = []
    for ticker in activos:
        r = log_ret[ticker].dropna()
        df_t = pd.DataFrame(index=r.index)

        # Retornos históricos
        df_t['log_ret_1d']  = r
        df_t['log_ret_5d']  = r.rolling(5).sum()
        df_t['log_ret_21d'] = r.rolling(21).sum()
        df_t['log_ret_63d'] = r.rolling(63).sum()

        # Momentum
        df_t['mom_1m']  = r.rolling(21).sum()
        df_t['mom_3m']  = r.rolling(63).sum()
```

```
        df_t['mom_6m']  = r.rolling(126).sum()
        df_t['mom_12m'] = r.rolling(252).sum()

        # Volatilidad
        df_t['vol_21d'] = r.rolling(21).std() * np.sqrt(252)
        df_t['vol_63d'] = r.rolling(63).std() * np.sqrt(252)

        # Sharpe histórico
        df_t['sharpe_63d'] = (r.rolling(63).mean() * 252 - RF_ANNUAL) /␣
↪df_t['vol_63d'].clip(1e-6)

        # RSI(14)
        delta = r.diff()
        gain  = delta.clip(lower=0).rolling(14).mean()
        loss  = (-delta.clip(upper=0)).rolling(14).mean()
        rs    = gain / loss.replace(0, np.nan)
        df_t['rsi_14'] = 100 - (100 / (1 + rs))

        # EMA ratios
        p = prices[ticker].dropna()
        ema20  = p.ewm(span=20,  adjust=False).mean()
        ema50  = p.ewm(span=50,  adjust=False).mean()
        ema200 = p.ewm(span=200, adjust=False).mean()
        df_t['ema_ratio_20_50']  = (ema20 / ema50)  - 1
        df_t['ema_ratio_50_200'] = (ema50 / ema200) - 1

        # Beta vs SPY
        if spy_ret is not None:
            cov_roll = r.rolling(63).cov(spy_ret)
            var_spy  = spy_ret.rolling(63).var()
            df_t['beta_63d'] = cov_roll / var_spy.replace(0, np.nan)

        # Target forward
        df_t['target_fwd'] = r.rolling(HORIZON).sum().shift(-HORIZON)

        df_t['Ticker'] = ticker
        frames.append(df_t.reset_index())

    ml_df_raw = pd.concat(frames, ignore_index=True)
    ml_df_raw['Date'] = pd.to_datetime(ml_df_raw['Date'])
    ml_df = ml_df_raw.set_index(['Date', 'Ticker']).
↪dropna(subset=['target_fwd'])

    FEATURE_COLS = [c for c in ml_df.columns if c not in ['target_fwd']]

    print(f'Dataset ML (construido): {ml_df.shape}')
    print(f'Features: {len(FEATURE_COLS)}: {FEATURE_COLS}')
```

```
print('\nSample del dataset ML:')
ml_df[FEATURE_COLS[:6] + ['target_fwd']].head(5)
```

[ ]:
```
#   Correlacion features vs target
corr_target = ml_df[FEATURE_COLS + ['target_fwd']].corr()['target_fwd'].
 ↪drop('target_fwd').sort_values()

fig, ax = plt.subplots(figsize=(10, max(4, len(corr_target)*0.3)))
colors = ['#E74C3C' if v < 0 else '#2ECC71' for v in corr_target]
corr_target.plot(kind='barh', ax=ax, color=colors, alpha=0.8)
ax.axvline(0, color='black', lw=0.8)
ax.set_title(f'Correlacion de Features vs Retorno Forward {HORIZON}d',␣
 ↪fontweight='bold')
ax.set_xlabel('Correlacion de Pearson')
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig(f'{OUTPUT_DIR}/correlacion_features_target.png', dpi=120,␣
 ↪bbox_inches='tight')
plt.show()

print('Top 5 features positivas:')
print(corr_target.tail(5).to_string())
print('\nTop 5 features negativas:')
print(corr_target.head(5).to_string())
```

1.7 3. Walk-Forward Cross-Validation Setup

[ ]:
```
def get_monthly_dates(df_index, start=None, end=None, freq='ME'):
    """
    Genera fechas de rebalanceo mensuales dentro del rango de datos.
    Retorna lista de Timestamps (ultimo dia habil de cada mes).
    """
    dates = df_index.get_level_values('Date').unique().sort_values()
    if start:
        dates = dates[dates >= pd.Timestamp(start)]
    if end:
        dates = dates[dates <= pd.Timestamp(end)]
    s = pd.Series(dates, index=dates)
    return pd.DatetimeIndex(s.groupby(dates.to_period('M')).max().values)


def walk_forward_predict(
    ml_df,
    feature_cols,
```

```
    model,
    train_end_initial = '2019-12-31',
    oos_start         = '2020-01-01',
    min_train_months  = 24,
    rebalance_freq    = 'ME',
):
    """
    Walk-Forward: en cada fecha de rebalanceo:
     1. Entrena con todos los datos hasta esa fecha (ventana expandida)
     2. Predice el retorno forward del mes siguiente
     3. Guarda predicciones y rankings

    Returns:
        predictions_df : DataFrame con (Date, Ticker, y_pred, y_true, rank)
    """
    all_dates = ml_df.index.get_level_values('Date').unique().sort_values()
    oos_dates = all_dates[all_dates >= pd.Timestamp(oos_start)]

    # Fechas de rebalanceo (ultimo dia bursatil de cada mes)
    s_dates   = pd.Series(oos_dates, index=oos_dates)
    reb_dates = pd.DatetimeIndex(s_dates.groupby(oos_dates.to_period('M')).
 ↪max().values)

    predictions = []

    for reb_date in reb_dates:
        # Datos disponibles hasta este punto (sin data leakage)
        cutoff = reb_date
        train_mask = ml_df.index.get_level_values('Date') <= cutoff
        test_mask  = ml_df.index.get_level_values('Date') == cutoff

        X_train = ml_df.loc[train_mask, feature_cols].dropna()
        y_train = ml_df.loc[train_mask, 'target_fwd'].dropna()

        # Interseccion de indices validos
        idx_valid = X_train.index.intersection(y_train.index)
        if len(idx_valid) < min_train_months * 20:
            continue

        X_tr = X_train.loc[idx_valid]
        y_tr = y_train.loc[idx_valid]

        # Test: prediccion para el siguiente periodo
        X_test = ml_df.loc[test_mask, feature_cols].dropna()
        y_test = ml_df.loc[test_mask, 'target_fwd'].dropna()
        idx_test = X_test.index.intersection(y_test.index)
```

```
        if len(idx_test) == 0:
            continue

        try:
            model.fit(X_tr.values, y_tr.values)
            y_pred = model.predict(X_test.loc[idx_test].values)
        except Exception as e:
            print(f'  Error en {reb_date}: {e}')
            continue

        tickers_test = idx_test.get_level_values('Ticker')
        for i, (idx, ticker) in enumerate(zip(idx_test, tickers_test)):
            predictions.append({
                'Date'  : reb_date,
                'Ticker': ticker,
                'y_pred': y_pred[i],
                'y_true': y_test.loc[idx] if idx in y_test.index else np.nan,
            })

    pred_df = pd.DataFrame(predictions)
    if len(pred_df) == 0:
        return pred_df

    # Ranking de predicciones (rank 1 = mejor prediccion)
    pred_df['rank'] = pred_df.groupby('Date')['y_pred'].rank(ascending=False,␣
 ↪method='first').astype(int)
    return pred_df

print('Funciones de walk-forward definidas')
print(f'  OOS a partir de: {OOS_START}')
print(f'  Horizonte forward: {HORIZON} dias')
print(f'  Top-K en portafolio: {TOP_K}')
```

[ ]:
```
def predictions_to_weights(pred_df, top_k=10, scheme='uniform'):
    """
    Convierte predicciones de ranking a pesos del portafolio.

    scheme:
      'uniform'  → 1/K para los top-K
      'score'    → pesos proporcionales al score predicho (clip a 0)
      'rank'     → pesos proporcionales inversos al rank (top=mayor peso)
    """
    weights_list = []
    for date, grp in pred_df.groupby('Date'):
        top = grp.nsmallest(top_k, 'rank').copy()
        if scheme == 'uniform':
            top['weight'] = 1.0 / len(top)
```

```
        elif scheme == 'score':
            scores = top['y_pred'].clip(lower=0)
            total  = scores.sum()
            top['weight'] = scores / total if total > 0 else 1 / len(top)
        elif scheme == 'rank':
            inv_rank = 1.0 / top['rank']
            top['weight'] = inv_rank / inv_rank.sum()
        top['Date'] = date
        weights_list.append(top[['Date', 'Ticker', 'weight']])

    return pd.concat(weights_list, ignore_index=True)


def backtest_portfolio(
    weights_df,
    log_ret,
    benchmark_ticker=BENCHMARK,
    horizon=21,
):
    """
    Backtesting mensual: aplica pesos del mes t a los retornos del mes t+1.
    Retorna serie de retornos diarios del portafolio.
    """
    portfolio_rets = []
    dates = weights_df['Date'].unique()
    dates = sorted(dates)

    for i, reb_date in enumerate(dates):
        next_reb = dates[i+1] if i + 1 < len(dates) else None

        # Pesos en este período
        w_row = weights_df[weights_df['Date'] == reb_date].
 ↪set_index('Ticker')['weight']

        # Período de aplicación: desde reb_date hasta next_reb
        if next_reb is not None:
            mask = (log_ret.index > reb_date) & (log_ret.index <= next_reb)
        else:
            mask = log_ret.index > reb_date

        ret_period = log_ret[mask]
        if len(ret_period) == 0:
            continue

        # Filtrar tickers disponibles
        tickers = [t for t in w_row.index if t in ret_period.columns]
        if len(tickers) == 0:
```

```
            continue

        w = w_row[tickers]
        w = w / w.sum()
        r_period = ret_period[tickers].dot(w)
        portfolio_rets.append(r_period)

    if not portfolio_rets:
        return pd.Series(dtype=float)

    return pd.concat(portfolio_rets).sort_index()

print('Funciones de backtesting definidas')
```

1.8 4a. Modelo: Ridge Regression

[ ]:
```
#   Limpiar features: drop inf y normalizar
ml_df_clean = ml_df[FEATURE_COLS + ['target_fwd']].replace([np.inf, -np.inf],␣
 ↪np.nan)

# Eliminar features con demasiados NAs
missing_rate = ml_df_clean.isna().mean()
keep_cols    = missing_rate[missing_rate < 0.3].index.tolist()
FEATURE_FINAL = [c for c in keep_cols if c != 'target_fwd']
print(f'Features finales (< 30% missing): {len(FEATURE_FINAL)}')

ml_df_clean = ml_df_clean[FEATURE_FINAL + ['target_fwd']].dropna()
print(f'Dataset limpio: {ml_df_clean.shape}')
```

[ ]:
```
print('Entrenando Ridge Regression (walk-forward)...')

ridge_model = Pipeline([
    ('scaler', RobustScaler()),
    ('ridge',  Ridge(alpha=10.0, fit_intercept=True)),
])

pred_ridge = walk_forward_predict(
    ml_df_clean,
    FEATURE_FINAL,
    ridge_model,
    train_end_initial = TRAIN_END_ML,
    oos_start         = OOS_START,
)
```

```
print(f'Predicciones Ridge: {len(pred_ridge)} filas | {pred_ridge["Date"].
 ↪nunique()} fechas')

if len(pred_ridge) > 0:
    mae  = mean_absolute_error(pred_ridge['y_true'].dropna(), pred_ridge.
 ↪loc[pred_ridge['y_true'].notna(), 'y_pred'])
    rmse = np.sqrt(mean_squared_error(pred_ridge['y_true'].dropna(), pred_ridge.
 ↪loc[pred_ridge['y_true'].notna(), 'y_pred']))
    ic   = pred_ridge.groupby('Date').apply(lambda g: g['y_pred'].
 ↪corr(g['y_true'])).mean()
    print(f'  MAE        : {mae:.5f}')
    print(f'  RMSE       : {rmse:.5f}')
    print(f'  IC (media) : {ic:.4f}')
```

[ ]:
```
#   Portafolio Ridge
if len(pred_ridge) > 0:
    weights_ridge  = predictions_to_weights(pred_ridge, top_k=TOP_K,␣
 ↪scheme='uniform')
    ret_ridge_port = backtest_portfolio(weights_ridge, log_ret)
    print(f'Portafolio Ridge - dias en backtest: {len(ret_ridge_port)}')
    spy_ret_oos = log_ret[log_ret.index >= OOS_START][BENCHMARK]
    m_ridge = full_metrics(ret_ridge_port, spy_ret_oos.reindex(ret_ridge_port.
 ↪index), label='Ridge')
    print(f"  Sharpe: {m_ridge['sharpe']:.3f} | MaxDD: {m_ridge['max_drawdown']:
 ↪.2%} | Ret anual: {m_ridge['ret_annual']:.2%}")
else:
    print('No se generaron predicciones Ridge')
```

1.9 4b. Modelo: Lasso Regression

[ ]:
```
print('Entrenando Lasso Regression (walk-forward)...')

lasso_model = Pipeline([
    ('scaler', RobustScaler()),
    ('lasso',  Lasso(alpha=0.001, max_iter=2000, fit_intercept=True)),
])

pred_lasso = walk_forward_predict(
    ml_df_clean,
    FEATURE_FINAL,
    lasso_model,
    train_end_initial = TRAIN_END_ML,
    oos_start         = OOS_START,
)
```

```
print(f'Predicciones Lasso: {len(pred_lasso)} filas | {pred_lasso["Date"].
 ↪nunique()} fechas')

if len(pred_lasso) > 0:
    mae  = mean_absolute_error(pred_lasso['y_true'].dropna(), pred_lasso.
 ↪loc[pred_lasso['y_true'].notna(), 'y_pred'])
    ic   = pred_lasso.groupby('Date').apply(lambda g: g['y_pred'].
 ↪corr(g['y_true'])).mean()
    print(f'  MAE: {mae:.5f} | IC: {ic:.4f}')

    weights_lasso  = predictions_to_weights(pred_lasso, top_k=TOP_K,␣
 ↪scheme='uniform')
    ret_lasso_port = backtest_portfolio(weights_lasso, log_ret)
    m_lasso = full_metrics(ret_lasso_port, spy_ret_oos.reindex(ret_lasso_port.
 ↪index), label='Lasso')
    print(f"  Sharpe: {m_lasso['sharpe']:.3f} | MaxDD: {m_lasso['max_drawdown']:
 ↪.2%} | Ret anual: {m_lasso['ret_annual']:.2%}")
```

1.10 4c. Modelo: Random Forest

[ ]:
```
print('Entrenando Random Forest (walk-forward)...')

rf_model = RandomForestRegressor(
    n_estimators  = 200,
    max_depth     = 6,
    min_samples_leaf = 20,
    max_features  = 'sqrt',
    random_state  = 42,
    n_jobs        = -1,
)

pred_rf = walk_forward_predict(
    ml_df_clean,
    FEATURE_FINAL,
    rf_model,
    train_end_initial = TRAIN_END_ML,
    oos_start         = OOS_START,
)

print(f'Predicciones RF: {len(pred_rf)} filas | {pred_rf["Date"].nunique()}␣
 ↪fechas')

if len(pred_rf) > 0:
    mae = mean_absolute_error(pred_rf['y_true'].dropna(), pred_rf.
 ↪loc[pred_rf['y_true'].notna(), 'y_pred'])
```

```
    ic  = pred_rf.groupby('Date').apply(lambda g: g['y_pred'].
↪corr(g['y_true'])).mean()
    print(f'  MAE: {mae:.5f} | IC: {ic:.4f}')

    weights_rf   = predictions_to_weights(pred_rf, top_k=TOP_K,␣
↪scheme='uniform')
    ret_rf_port  = backtest_portfolio(weights_rf, log_ret)
    m_rf = full_metrics(ret_rf_port, spy_ret_oos.reindex(ret_rf_port.index),␣
↪label='RandomForest')
    print(f"  Sharpe: {m_rf['sharpe']:.3f} | MaxDD: {m_rf['max_drawdown']:.2%}␣
↪| Ret anual: {m_rf['ret_annual']:.2%}")
```

[ ]:
```
#   Feature importance (Random Forest)
if len(pred_rf) > 0:
    # Reentrenar con todos los datos IS para obtener importancias estables
    X_all = ml_df_clean[FEATURE_FINAL].dropna()
    y_all = ml_df_clean.loc[X_all.index, 'target_fwd']
    idx   = X_all.index.intersection(y_all.index)

    rf_full = RandomForestRegressor(n_estimators=200, max_depth=6,␣
↪min_samples_leaf=20, random_state=42, n_jobs=-1)
    rf_full.fit(X_all.loc[idx].values, y_all.loc[idx].values)

    importance_df = pd.Series(rf_full.feature_importances_,␣
↪index=FEATURE_FINAL).sort_values(ascending=True)

    fig, ax = plt.subplots(figsize=(10, max(4, len(importance_df)*0.3)))
    importance_df.plot(kind='barh', ax=ax, color='#27AE60', alpha=0.85)
    ax.set_title('Random Forest - Feature Importance (MDI)', fontweight='bold')
    ax.set_xlabel('Importancia relativa')
    ax.grid(axis='x', alpha=0.3)
    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/rf_feature_importance.png', dpi=120,␣
↪bbox_inches='tight')
    plt.show()
    print('Top 10 features por importancia:')
    print(importance_df.tail(10).to_string())
```

1.11 4d. Modelo: XGBoost

[ ]:
```
print('Entrenando XGBoost (walk-forward)...')

xgb_model = xgb.XGBRegressor(
    n_estimators      = 300,
    max_depth         = 4,
```

```
    learning_rate     = 0.05,
    subsample         = 0.8,
    colsample_bytree  = 0.7,
    min_child_weight  = 20,
    reg_alpha         = 0.1,
    reg_lambda        = 1.0,
    random_state      = 42,
    n_jobs            = -1,
    verbosity         = 0,
)

pred_xgb = walk_forward_predict(
    ml_df_clean,
    FEATURE_FINAL,
    xgb_model,
    train_end_initial = TRAIN_END_ML,
    oos_start         = OOS_START,
)

print(f'Predicciones XGB: {len(pred_xgb)} filas | {pred_xgb["Date"].nunique()}␣
 ↪fechas')

if len(pred_xgb) > 0:
    mae = mean_absolute_error(pred_xgb['y_true'].dropna(), pred_xgb.
 ↪loc[pred_xgb['y_true'].notna(), 'y_pred'])
    ic  = pred_xgb.groupby('Date').apply(lambda g: g['y_pred'].
 ↪corr(g['y_true'])).mean()
    print(f'  MAE: {mae:.5f} | IC: {ic:.4f}')

    weights_xgb   = predictions_to_weights(pred_xgb, top_k=TOP_K,␣
 ↪scheme='uniform')
    ret_xgb_port  = backtest_portfolio(weights_xgb, log_ret)
    m_xgb = full_metrics(ret_xgb_port, spy_ret_oos.reindex(ret_xgb_port.index),␣
 ↪label='XGBoost')
    print(f"  Sharpe: {m_xgb['sharpe']:.3f} | MaxDD: {m_xgb['max_drawdown']:.
 ↪2%} | Ret anual: {m_xgb['ret_annual']:.2%}")
```

[ ]:
```
#   XGBoost Feature Importance (gain)
if len(pred_xgb) > 0:
    X_all = ml_df_clean[FEATURE_FINAL].dropna()
    y_all = ml_df_clean.loc[X_all.index, 'target_fwd']
    idx   = X_all.index.intersection(y_all.index)

    xgb_full = xgb.XGBRegressor(
        n_estimators=300, max_depth=4, learning_rate=0.05,
        subsample=0.8, colsample_bytree=0.7, min_child_weight=20,
        random_state=42, n_jobs=-1, verbosity=0
```

```
    )
    xgb_full.fit(X_all.loc[idx].values, y_all.loc[idx].values)

    xgb_imp = pd.Series(xgb_full.feature_importances_, index=FEATURE_FINAL).
 ↪sort_values(ascending=True)

    fig, ax = plt.subplots(figsize=(10, max(4, len(xgb_imp)*0.3)))
    xgb_imp.plot(kind='barh', ax=ax, color='#E74C3C', alpha=0.85)
    ax.set_title('XGBoost - Feature Importance (Gain)', fontweight='bold')
    ax.set_xlabel('Importancia relativa')
    ax.grid(axis='x', alpha=0.3)
    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/xgb_feature_importance.png', dpi=120,␣
 ↪bbox_inches='tight')
    plt.show()
    print('Top 10 features XGBoost:')
    print(xgb_imp.tail(10).to_string())
```

1.12 5. Análisis del IC (Information Coefficient)

[ ]:
```
#   IC mensual por modelo
ic_results = {}
for name, pred_df in [('Ridge', pred_ridge), ('Lasso', pred_lasso), ('RF',␣
 ↪pred_rf), ('XGBoost', pred_xgb)]:
    if len(pred_df) == 0:
        continue
    ic_monthly = pred_df.groupby('Date').apply(
        lambda g: g['y_pred'].corr(g['y_true'])
    ).dropna()
    ic_results[name] = ic_monthly

if ic_results:
    ic_df = pd.DataFrame(ic_results)

    fig, axes = plt.subplots(2, 1, figsize=(13, 8), sharex=True)

    # IC rolling
    for col in ic_df.columns:
        color = MODEL_COLORS.get(col, '#999')
        axes[0].plot(ic_df.index, ic_df[col].rolling(3).mean(), lw=1.6,␣
 ↪label=col, color=color)
    axes[0].axhline(0, color='black', lw=0.8, linestyle='--')
    axes[0].set_ylabel('IC Rolling 3M')
    axes[0].set_title('Information Coefficient (IC) mensual por modelo',␣
 ↪fontweight='bold')
```

```
    axes[0].legend(fontsize=9)
    axes[0].grid(True, alpha=0.2)

    # IC acumulado
    for col in ic_df.columns:
        color = MODEL_COLORS.get(col, '#999')
        axes[1].plot(ic_df.index, ic_df[col].cumsum(), lw=1.6, label=col,␣
↪color=color)
    axes[1].axhline(0, color='black', lw=0.8, linestyle='--')
    axes[1].set_ylabel('IC Acumulado')
    axes[1].set_title('IC Acumulado (mayor es mejor)', fontweight='bold')
    axes[1].legend(fontsize=9)
    axes[1].grid(True, alpha=0.2)

    axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/ic_mensual_modelos.png', dpi=120,␣
↪bbox_inches='tight')
    plt.show()

    print('\nResumen IC por modelo:')
    print(ic_df.agg(['mean','std','min','max']).round(4).to_string())
```

1.13 6. Comparativa de NAV y Métricas OOS

[ ]:
```
#   Equal Weight baseline
activos_oos = [t for t in ALL_TICKERS if t in log_ret.columns]
ew_weights  = pd.Series(1.0/len(activos_oos), index=activos_oos)
ret_ew_oos  = log_ret[log_ret.index >= OOS_START][activos_oos].dot(ew_weights)
spy_ret_oos = log_ret[log_ret.index >= OOS_START][BENCHMARK]

m_ew  = full_metrics(ret_ew_oos, spy_ret_oos, label='EW')
m_spy = full_metrics(spy_ret_oos, spy_ret_oos, label='SPY')

print(f"EW  - Sharpe: {m_ew['sharpe']:.3f} | Ret: {m_ew['ret_annual']:.2%} |␣
↪MDD: {m_ew['max_drawdown']:.2%}")
print(f"SPY - Sharpe: {m_spy['sharpe']:.3f} | Ret: {m_spy['ret_annual']:.2%} |␣
↪MDD: {m_spy['max_drawdown']:.2%}")
```

[ ]:
```
#   Tabla de métricas consolidada
all_metrics = {
    'EW (baseline)': m_ew,
    'SPY':           m_spy,
}
ret_series = {
```

```
    'EW (baseline)': ret_ew_oos,
    'SPY':           spy_ret_oos,
}

for name, pred_df, m in [
    ('Ridge',        pred_ridge, m_ridge if 'pred_ridge' in dir() and␣
 ↪len(pred_ridge) > 0 else None),
    ('Lasso',        pred_lasso, m_lasso if 'pred_lasso' in dir() and␣
 ↪len(pred_lasso) > 0 else None),
    ('RandomForest', pred_rf,    m_rf    if 'pred_rf'    in dir() and␣
 ↪len(pred_rf)    > 0 else None),
    ('XGBoost',      pred_xgb,   m_xgb   if 'pred_xgb'  in dir() and␣
 ↪len(pred_xgb)   > 0 else None),
]:
    if m is not None:
        all_metrics[name] = m

metrics_df = pd.DataFrame(all_metrics).T
display_cols =␣
 ↪['ret_annual','vol_annual','sharpe','sortino','max_drawdown','cvar_95','calmar','informatio

print('='*80)
print('TABLA COMPARATIVA OOS - ML vs Baselines')
print('='*80)
print(metrics_df[display_cols].round(4).to_string())
```

[ ]:
```
#   NAV acumulado comparativo
nav_series = {}
nav_series['EW (baseline)'] = (1 + ret_ew_oos).cumprod() * 100
nav_series['SPY']           = (1 + spy_ret_oos).cumprod() * 100
for name, ret_port in [
    ('Ridge',        ret_ridge_port if 'ret_ridge_port' in dir() else None),
    ('Lasso',        ret_lasso_port if 'ret_lasso_port' in dir() else None),
    ('RandomForest', ret_rf_port    if 'ret_rf_port'    in dir() else None),
    ('XGBoost',      ret_xgb_port   if 'ret_xgb_port'  in dir() else None),
]:
    if ret_port is not None and len(ret_port) > 0:
        nav_series[name] = (1 + ret_port).cumprod() * 100
fig = plt.figure(figsize=(14, 9))
gs  = GridSpec(3, 1, height_ratios=[3, 1, 1], hspace=0.08)
ax_nav = fig.add_subplot(gs[0])
ax_dd  = fig.add_subplot(gs[1], sharex=ax_nav)
ax_vol = fig.add_subplot(gs[2], sharex=ax_nav)
for name, nav in nav_series.items():
    color = MODEL_COLORS.get(name, '#888')
    lw    = 1.5 if name not in ['SPY', 'EW (baseline)'] else 1.2
    ls    = '--' if name in ['SPY', 'EW (baseline)'] else '-'
```

```
    ax_nav.plot(nav, color=color, lw=lw, linestyle=ls, label=name)
ax_nav.axhline(100, color='black', lw=0.6, linestyle=':')
ax_nav.axvspan(pd.Timestamp('2020-01-01'), pd.Timestamp('2020-12-31'), alpha=0.
 ↪07, color='red',  label='Pandemia 2020')
ax_nav.axvspan(pd.Timestamp('2023-01-01'), pd.Timestamp('2026-12-31'), alpha=0.
 ↪05, color='blue', label='Eval OOS Final')
ax_nav.set_ylabel('Valor Acumulado (base 100)')
ax_nav.set_title('Portafolios ML vs Baselines - Backtest OOS (2020-2025)',␣
 ↪fontweight='bold')
ax_nav.legend(loc='upper left', fontsize=8.5, ncol=2)
ax_nav.grid(True, alpha=0.2)
# Drawdown del mejor modelo ML
best_nav_name = max(
    [n for n in nav_series if n not in ['SPY', 'EW (baseline)']],
    key=lambda n: metrics_df.loc[n, 'sharpe'] if n in metrics_df.index else -99,
    default=None
)
if best_nav_name and best_nav_name in nav_series:
    nav_best = nav_series[best_nav_name]
    dd_best  = (nav_best - nav_best.cummax()) / nav_best.cummax() * 100
    ax_dd.fill_between(dd_best.index, dd_best, 0, color=MODEL_COLORS.
 ↪get(best_nav_name, '#888'), alpha=0.4, label=best_nav_name)
nav_spy = nav_series['SPY']
dd_spy  = (nav_spy - nav_spy.cummax()) / nav_spy.cummax() * 100
ax_dd.fill_between(dd_spy.index, dd_spy, 0, color=MODEL_COLORS.get('SPY',␣
 ↪'#95A5A6'), alpha=0.25, label='SPY')
ax_dd.set_ylabel('Drawdown (%)')
ax_dd.legend(loc='lower left', fontsize=8)
ax_dd.grid(True, alpha=0.2)
# Volatilidad rolling
for name, nav in nav_series.items():
    ret_tmp = nav.pct_change().dropna()
    vol_tmp = ret_tmp.rolling(21).std() * np.sqrt(252) * 100
    ax_vol.plot(vol_tmp, color=MODEL_COLORS.get(name, '#888'), lw=1.1, alpha=0.
 ↪85, label=name)
ax_vol.set_ylabel('Vol. 21d (%)')
ax_vol.legend(loc='upper left', fontsize=7, ncol=2)
ax_vol.grid(True, alpha=0.2)
ax_vol.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.setp(ax_nav.get_xticklabels(), visible=False)
plt.setp(ax_dd.get_xticklabels(),  visible=False)
plt.savefig(f'{OUTPUT_DIR}/nav_comparativo_ml.png', dpi=150,␣
 ↪bbox_inches='tight')
plt.show()
print('Grafico guardado')
```

1.14 7. Análisis de Composición: ¿Qué activos seleccionan los modelos?

[ ]:
```
#   Frecuencia de seleccion por ticker y modelo
selection_stats = {}
for name, pred_df in [('Ridge', pred_ridge), ('Lasso', pred_lasso), ('RF',␣
 ↪pred_rf), ('XGBoost', pred_xgb)]:
    if len(pred_df) == 0:
        continue
    top_selections = pred_df[pred_df['rank'] <= TOP_K]
    freq = top_selections.groupby('Ticker').size() / pred_df['Date'].nunique()
    selection_stats[name] = freq

if selection_stats:
    sel_df = pd.DataFrame(selection_stats).fillna(0)
    sel_df['Sector'] = sel_df.index.map(lambda t: SECTOR_LOOKUP.get(t,␣
 ↪'Unknown'))

    fig, ax = plt.subplots(figsize=(13, max(6, len(sel_df)*0.35)))
    sel_df_plot = sel_df.drop(columns=['Sector']).
 ↪sort_values(by=list(selection_stats.keys()), ascending=True)
    bar_colors  = [PALETTE.get(SECTOR_LOOKUP.get(t, ''), '#aaa') for t in␣
 ↪sel_df_plot.index]

    x = np.arange(len(sel_df_plot))
    width = 0.2
    for i, col in enumerate(sel_df_plot.columns):
        ax.barh(x + i*width, sel_df_plot[col].values, width,
                label=col, color=MODEL_COLORS.get(col, '#888'), alpha=0.8)

    ax.set_yticks(x + width)
    ax.set_yticklabels(sel_df_plot.index, fontsize=8)
    ax.set_xlabel('Frecuencia de seleccion (fraccion de meses)')
    ax.set_title(f'Frecuencia de Seleccion en Top-{TOP_K} por Modelo',␣
 ↪fontweight='bold')
    ax.legend(fontsize=9)
    ax.grid(axis='x', alpha=0.3)
    ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0))

    plt.tight_layout()
    plt.savefig(f'{OUTPUT_DIR}/frecuencia_seleccion.png', dpi=120,␣
 ↪bbox_inches='tight')
    plt.show()

    print('\nTop 10 activos mas seleccionados (promedio entre modelos):')
    avg_sel = sel_df.drop(columns=['Sector']).mean(axis=1).
 ↪sort_values(ascending=False)
    print(avg_sel.head(10).to_string())
```

1.15 8. Comparativa Final vs Markowitz

[ ]:
```
#   Cargar resultados Markowitz para comparar
mkw_prepandemic_dir = '../data/processed/markowitz_prepandemic'
mkw_orig_dir        = '../data/processed/markowitz'

comparison_rows = {}

# Nuestros modelos ML
for name in all_metrics:
    comparison_rows[name] = all_metrics[name]

# Markowitz pre-pandemia
try:
    nav_mkw = pd.read_parquet(f'{mkw_prepandemic_dir}/nav_oos_prepandemia.
 ↪parquet')
    spy_full = log_ret[BENCHMARK] if BENCHMARK in log_ret.columns else pd.
 ↪Series(dtype=float)
    for col in ['MSR', 'GMV', 'EW']:
        if col in nav_mkw.columns:
            ret_mkw = nav_mkw[col].pct_change().dropna()
            spy_aligned = spy_full.reindex(ret_mkw.index)
            m = full_metrics(ret_mkw, spy_aligned, label=f'Mkw_{col}')
            comparison_rows[f'Markowitz-{col}'] = m
    print('Resultados Markowitz pre-pandemia cargados')
except FileNotFoundError:
    print('No se encontraron resultados Markowitz pre-pandemia (ejecutar 03b␣
 ↪primero)')

# Tabla final
comp_df = pd.DataFrame(comparison_rows).T
display_cols =␣
 ↪['ret_annual','vol_annual','sharpe','sortino','max_drawdown','cvar_95','calmar']

print('\n' + '='*80)
print('COMPARATIVA FINAL: ML vs Markowitz vs SPY')
print('='*80)
print(comp_df[display_cols].round(4).sort_values('sharpe', ascending=False).
 ↪to_string())
```

[ ]:
```
#   Gráfico de burbujas: Retorno vs Riesgo vs Sharpe
fig, ax = plt.subplots(figsize=(11, 7))

for model_name, row in comp_df.iterrows():
    ret  = row.get('ret_annual', np.nan)
```

```
    vol  = row.get('vol_annual', np.nan)
    sharpe = row.get('sharpe', 0)
    if np.isnan(ret) or np.isnan(vol):
        continue

    color = MODEL_COLORS.get(model_name, '#888888')
    if 'Markowitz' in model_name or model_name in ['EW (baseline)', 'SPY']:
        marker = 'D'
        alpha  = 0.6
    else:
        marker = 'o'
        alpha  = 0.9

    size = max(50, abs(sharpe) * 200)
    ax.scatter(vol, ret, s=size, c=color, marker=marker, alpha=alpha,
               edgecolors='white', linewidths=1.2, zorder=5)
    ax.annotate(model_name, (vol, ret), xytext=(6, 4),
                textcoords='offset points', fontsize=8, color=color)

ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_xlabel('Volatilidad Anualizada')
ax.set_ylabel('Retorno Anualizado OOS')
ax.set_title('Mapa Riesgo-Retorno: ML vs Markowitz vs SPY (OOS 2020-2025)',␣
 ↪fontweight='bold')
ax.grid(True, alpha=0.2, linestyle='--')

legend_elements = [
    plt.scatter([], [], marker='o', color='#888', s=100, label='Modelos ML'),
    plt.scatter([], [], marker='D', color='#888', s=100, label='Baselines /␣
 ↪Markowitz'),
]
ax.legend(handles=legend_elements, loc='upper left', fontsize=9)

plt.tight_layout()
plt.savefig(f'{OUTPUT_DIR}/mapa_riesgo_retorno_comparativa.png', dpi=150,␣
 ↪bbox_inches='tight')
plt.show()
print('Grafico guardado')
```

1.16 9. Persistencia de Resultados

[ ]:
```
#   Guardar métricas, predicciones, pesos y modelos
import joblib
```

```python
# 1. Tablas de métricas
comp_df.to_csv(f'{OUTPUT_DIR}/comparativa_ml_markowitz.csv')
metrics_df.to_csv(f'{OUTPUT_DIR}/metrics_ml_modelos.csv')

# 2. Predicciones históricas (Date, Ticker, y_pred, y_true, rank)
for name, pred_df in [('ridge', pred_ridge), ('lasso', pred_lasso), ('rf', pred_rf), ('xgb', pred_xgb)]:
    if len(pred_df) > 0:
        pred_df.to_parquet(f'{OUTPUT_DIR}/predictions_{name}.parquet', index=False)

# 3. Pesos dinámicos del portafolio mes a mes (Date, Ticker, weight)
for name, w_df in [('ridge', weights_ridge if 'weights_ridge' in dir() else None),
                   ('lasso', weights_lasso if 'weights_lasso' in dir() else None),
                   ('rf',    weights_rf    if 'weights_rf'    in dir() else None),
                   ('xgb',   weights_xgb   if 'weights_xgb'   in dir() else None)]:
    if w_df is not None and len(w_df) > 0:
        w_df.to_parquet(f'{OUTPUT_DIR}/weights_{name}.parquet', index=False)

# 4. Curvas de NAV acumulado
nav_export = pd.DataFrame(nav_series)
nav_export.to_parquet(f'{OUTPUT_DIR}/nav_ml_modelos.parquet')

# 5. Importancia de features
if 'importance_df' in dir():
    importance_df.to_csv(f'{OUTPUT_DIR}/rf_feature_importance.csv')
if 'xgb_imp' in dir():
    xgb_imp.to_csv(f'{OUTPUT_DIR}/xgb_feature_importance.csv')

# 6. Guardar los modelos entrenados finales (artefactos para inferencia / NB 07)
try:
    joblib.dump(ridge_model, f'{OUTPUT_DIR}/model_ridge_final.joblib')
    joblib.dump(lasso_model, f'{OUTPUT_DIR}/model_lasso_final.joblib')
    joblib.dump(rf_model,    f'{OUTPUT_DIR}/model_rf_final.joblib')
    xgb_model.save_model(f'{OUTPUT_DIR}/model_xgb_final.json')
    print('Artefactos de modelos (.joblib / .json) guardados correctamente.')
except Exception as e:
    print(f'Aviso al guardar modelos binarios: {e}')

print(f'\nTodos los resultados guardados en: {OUTPUT_DIR}/')
for f in sorted(os.listdir(OUTPUT_DIR)):
    fpath = os.path.join(OUTPUT_DIR, f)
```

```
    size  = os.path.getsize(fpath) / 1024
    print(f'  {f:40s} {size:>8.1f} KB')
```

1.17 10. Conclusiones

1.17.1 Hallazgos

Modelo	Fortaleza	Debilidad
Ridge	Estable, baja varianza, buena base	Limitado a relaciones lineales
Lasso	Seleccion de features, interpretable	Puede ser inestable con multicolinealidad
Random Forest	Captura no-linealidades, robusto	Lento para refit frecuente, overfitting con NAs
XGBoost	Alto poder predictivo, maneja outliers	Riesgo de overfitting en ventanas cortas

1.17.2 Metricas clave

- **IC $>$ 0.03** indica senales estadisticamente utiles para ranking
- **Sharpe $>$ SPY** valida la adicion de valor del modelo ML
- **Max Drawdown $<$ Markowitz GMV** es el objetivo de proteccion ante shocks

1.17.3 Proximos pasos

1. `07_cvar_portfolio_opt.ipynb` — Optimizacion CVaR con los scores de ML como retornos esperados
2. Ajuste de hiperparametros con walk-forward CV mas granular
3. Ensemble de modelos (combinacion lineal de rankings)
4. Inclusion de features macroeconomicas (VIX, curva de tasas)